rogii-wellbore-geology-prediction

26位 Solution - Stacked UNet + Synthetic Well Pretraining

Rank	26
Team	Tucker Arrants
Author	Tucker Arrants
Topic ID	733136
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/rogii-wellbore-geology-prediction/discussion/733136
Retrieved with Kaggle CLI on 2026-08-06.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

主催者と参加者の皆様に感謝します。難しいコンペで、shake-up後にgold zoneへ残れなかったのは残念ですが、数か月で多くを学び、他の方々のSolutionを読むのを楽しみにしています。

TL;DR

各wellをimageとして扱います。rowsはbucket化したTVT depths、columnsはbucket化したMD stepsです。vertical axisはlast known TVT（projection-start anchor）を中心とする固定256 ft windowで、H bins（main modelは256×1.0 ft、diversity memberの1つは224×1.14 ft）へ分割します。horizontal axisはwellの全MD rangeを`W = 2H` equal columnsへ分け、1 columnが連続する数log rowsをaggregateします。stacked UNetがこのimageを受け、MD columnごとにdepth rows上のsoftmax distributionを出し、binned true TVTへのcross-entropyでtrainingします。final predictionはdistributionのexpected value（soft-argmax）で、classification headからsub-bin continuous depthを得ます。1 forward passでwell全体を予測します。

5 input channels:

- **Typewell GR**: typewell logをrow gridへinterpolateし、各rowに「このTVTでstratigraphy上あるべきGR」を持たせる。column方向へconstant、wellごとにz-score。
- **Horizontal GR**: lateral GRをMD columnごとにaverageし、各columnに「bitがここで実測したGR」を持たせる。row方向へconstant、wellごとにz-score。
- **Hint channel**: known pre-projection-start trajectoryをone-hot curveでrasterize。labeled columnのtrue depth rowだけ点灯し、projection start後はzero。predictionをanchorし、curveをゼロから探さずvisible curveのcontinuationにする。
- **Cost volume**: channels 1/2の各`(row,column)`でのsquared difference。dark valleyは「このMDのobserved GRがtypewellのこのdepthのGRと一致」を意味するfull correlation surface。古典的geosteering cross-correlation panelをhand decodeせずfeatureとして渡す。
- **dZ channel**: columnごとのmean `dZ/dMD`（borehole local vertical gradient / inclination proxy）。z-scoreしてclipし、row方向へconstant。well自身がframe内をどれだけ急に動くかを伝える。

直感的にはchannel 4がwellのpossible positionを描き、channel 3が過去位置をpinし、channels 1-2はhand-built cost surfaceをnetworkが再判断するraw ingredient、channel 5はtrajectory physicsです。modelはhintからcost-volume ridgeをforward trackするので、per-column classification head付きsegmentation backboneが自然に合います。

Model Specifications

SMPの標準ImageNet-pretrained ResNet34から始めました。CV 7.5付近のwallでlarger backbonesを試し、capacityだけで性能が上がるかEffNet B7/B8へ一気に拡大すると上がりました。次にEffNetV2 Large、XL、最後に最大EffNetであるL2へ進み、stack時は約10億parametersでした。可能にしたrecipeは`larger backbone → bigger pretrain size → heavier augmentations`です。性能向上はXLでplateauしましたが、LとXLのcorrelationが高すぎるためdiversity目的でL2をtrainingしました。B7/B8はCV約5.8と悪い割にdecorrelateせずfinal ensembleからdropしました。

それ以外は標準SMP UNet classで、decoder channelsだけを `(256,128,64,32,16)` から `(512,256,128,64,32)` へ拡大し、残りはdefaultです。

Stacking / Re-feeding

capacity増加でCVが下がるためstacked UNetを試しました。Stage 1は5-channel imageからfirst per-column depth distributionを出します。Stage 2は同じ5 channelsにStage 1のsoftened outputを6th channelとして加えます。Stage 1がtrajectoryをどこだと信じるかを描く「belief plane」です。

handoffはtemperature `T=1.5` でsoftenし、2 UNets間gradientはdetachします。gradientを流すとoverfitしました。Stage 1は同じtargetへのown cross-entropy（half weight）でtrainingし、Stage 2 lossは触れません。各stageは独立にtrajectoryを予測し、Stage 2はStage 1の意見も見ます。このideaはYale/UNC-CH competitionのRuby's solutionから得ました。

inferenceが興味深い点です。Stage 2はbelief planeを消費するようtrainingされているため、自分のoutputを再び入力できます。predict、soften、re-feedを反復し、`k=2` でさらに改善しました。training中もprobability 0.5でmulti-round in-graphを実行して複数belief refinementを学ばせ、`k=3` まで有効、以降は悪化しました。

self-refeedによりtrajectory estimateとGR evidenceを再度調停できます。EffNetV2 L/XLではoptimal `k=3`。L2ではrefeedが明確に悪く、`k>1` ごとに急速にoverfitしました。

Pretraining on Synthetic Wells

real training dataは773 wellsだけで、standard pretrained CNNには小さいため、全modelをsynthetic wellsでpretrainしてからrealでfine-tuneします。初期ResNet34はlow loss weightでsyntheticをjoint trainingしましたがlarger modelには効かず、pretrainingがbestでした。同じpretrained modelから複数fine-tuneでき時間も節約します。

synthetic wellはreal typewell columnから始めます。stratigraphyを借り、fake lateralをdrillします。layer内を上下するgenerated trajectoryにgeological events（faults、unconformities）、measurement noise、calibration driftを加え、同じimage pipelineでrenderすれば無限の `(well image, true depth curve)` pairsができます。

brute-force experimentで、synthetic dataはreal dataを模倣すべきでないと学びました。synthetic/realのstatistical gapを閉じるほど、ほぼ毎回悪化しました。有効だったcurriculumは、geologyをrealよりhard（large fault throws、aggressive trajectories）、matching signalをrealよりclean（lateral/typewell間のcrisp GR correspondence）にすることです。synthetic worldの仕事は技能を教えることで、realism testに合格することではありません。

synthetic poolはfoldごとにそのfoldのtraining typewells/geologyだけから生成し、validation wellsはpretrainingでもunseenです。deployed modelは全training wellsからpoolを作り、test scaleでも同じruleを適用します。厳密に必要かは不明ですが、追加pretraining costを払って安全側にしました。各seedも独自pretraining phaseを持ち、「lucky」pretrained weightsを全fine-tuneへ継承しないようにしました。

Augmentations

real wellsが少ないためfine-tuneはaugmentationを強く使います。すべてraw well上でimage rendering前に適用し、rendered imageのself-consistencyを保ちます。

- **MD warp**: well length方向をsmoothにstretch/compressし、同じgeologyを異なるlateral scaleへ。
- **Bed warp**: typewellのbed thicknessをsmoothにperturbし、exact layer spacingのmemorizeを防ぐ。
- **Horizontal flip**: well directionをreverseし、physical consistencyのためdZ signもflip。
- **Hint drop**: known-trajectory hintの一部をrandom deleteし、anchorだけに頼らずGR correlationを読ませる。
- **Anchor jitter**: last known TVTに対するvertical windowをrandom offsetし、trajectoryがcenter rowから始まると学習せずhintからanchorを探させる。

trainingではEMA（decay 0.997）をactual checkpointにします。raw weightsはnoisyでaveraged copyが常に良好です。regularizationのためbatch size 8も使いました。

Seed Noise

当初からseedでCVが大きく変わりtrainingもvolatileでした。pretraining、EMA、augmentationでspreadは狭まりましたが、最終的にfoldごと7 seedsをensembleしました。boostは5 seedsでplateauしましたが、安心のため2つ追加しました。foldのseed-model varianceに応じ、mean modelに対し0.2-0.6 ftのensemble boostでした。

EffNetsも有効でした。他のlarger backbonesはhealthy trainingにならずEffNet familyへ留まりました。

CV Results

CVはwellでgroupedし、bucketed TVT rangeでstratifyしてfold difficultyをbalanceしました。TVT range（anchorからwellがどれだけ外れるか）はdifficultyのstrong predictorで、773 wellsを5 foldsへunstratifiedに分けるとhard wellsが1 foldへ固まり、model changeでなくluckでscoreが動きます。

Discussionで適切なmodelならsub-5 ft RMSEと書いた時、best model CVは4.98でした。しかし1 foldがluckyに見えたためretrainし、「fairer」に見えるweightsを選ぶとCV 5.136になりました。これがsingle-modelで到達したlowestです。この種のluckを扱った経験がなく、principledな判断だったかは分かりません。

Final Ensemble

large-capacity UNetsでもwell-image approachはinferenceが比較的速く、多数ensembleできました。

- Model 1: EffNetV2 L、`k=3`、5 folds×7 seeds、CV 5.157
- Model 2: EffNetV2 L（smaller image）、`k=3`、5 folds×7 seeds、CV 5.252
- Model 3: EffNetV2 XL、`k=3`、5 folds×7 seeds、CV 5.136
- Model 4: EffNet L2、`k=1`、5 folds×7 seeds、CV 5.26

optimized weightsの改善はわずかでseed noiseへのoverfitを避けるためequal-weight ensembleにしました。final CV 5.038、Private LB 6.66でした。

最後の週にGRUをtrainingしましたがCV 7.8を切れませんでした。同じTVT/MD binning + CE lossはHuber direct regressionより良く、stackも有効でしたが、最終的に捨てました。そのためfinal ensembleは特にdecorrelateしておらず、よりcompetitiveなnon-UNet modelがあればさらに伸ばせたでしょう。

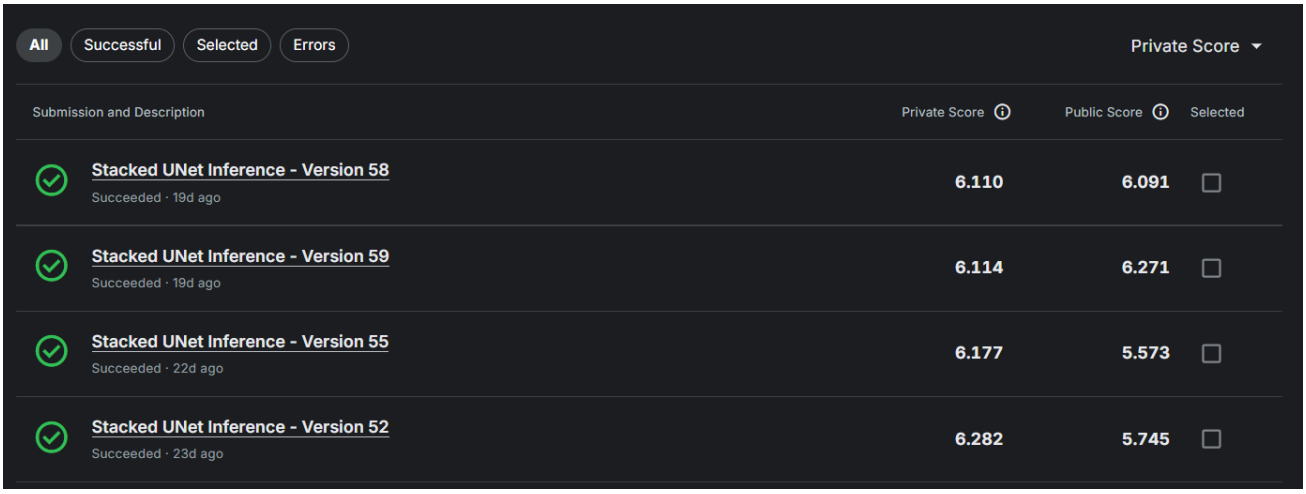
What Did Not Work / Was Unused

- formation top data、X/Y coordinatesは未使用
- BAN-style self distillation
- EfficientNet family以外のbackbone
- auxiliary training tasks
- vanilla CE以外のloss modification（soft CE、MD-weighted CEなど）
- postprocessing/TTA
- PixelShuffle

Neighbor Well Dataを使う未選択Gold Submission

数週間前にspatial-neighbor prior based UNetを作りました。各training wellのpost-PS driftを16 segmentsのpiecewise-linear slopesへfitし、そのwell headingに沿うapparent dipをazimuth normalizationでtrue structural dipへ変換してmap-wide drift fieldへ入れます。new wellには約50 nearest field samplesのlocal-linear regressionで各segmentのtrue dipを推定し、own headingへprojectしpre-PS anchorからintegrateします。standalone priorは11.8 ftでした。このideaはこちらの素晴らしいnotebookに全面的に触発されました。

Stage 2 modelの7th input channelとして入れ、Gaussian belief bumpをnetworkがgateしました。best single model CV 5.027でしたが、fold splitがspatial-basedでなくCV leakageと思いdeployから外しました。厳しいlessonです。下のscreenshot submissionsはさまざまなsingle modelでこのneighbor spatial dataを使っています。



Links: Synth generator for pretraining

補足Q&A

Q1: Segmentation headとdecode

MarkDjadchenko（質問）: 最初のversionではsegmentationをそのまま使いましたか、それとも何らかのprocessingをしましたか？

Tucker Arrants（回答）: UNet headは常にMD columnごとのdepth rowsへのcross-entropy（1 row = 1 class）で、BCEのbinary maskではありませんでした。decodeもargmaxではなく、常に各column softmaxのexpected value（soft-argmax）です。segmentationのliteralnessとはこの意味だと思います。

model自体はコンペ最初の数週間後に持っていたものと全く同じで、preliminary resultsはこちらで共有しました。pretraining、model、augmentationをscaleし、一般的なhyperparameter tuningを行っただけです。

Q2: 5 channelsとaugmentationの重要度

MarkDjadchenko（質問）: 私は`abs(GR Typewell - GR Horizontal)`のcost mapだけを入力した似たmodelをtrainingし、U-Netは14 LB/CVでした。augmentationsが重要だったのか、5 channelsすべてがkeyだったのか知りたいです。

Tucker Arrants（回答）: 5 channelsすべてがkeyでした。`dZ` channelなし、つまりGR matching channelsだけでは約11.5 CVでした。

pretrainingもaugmentationとともに非常に重要ですが、最重要はpretrainingだと思います。synthetic poolの変更とsize増加だけでCVを数ft削れました。

Q3: Training compute

Universal Function Approximator（質問）: synthetic well dataでU-Netをpretrainしたとのことですが、Kaggle notebooksだけで行いましたか、それともpersonal compute environmentですか？

Tucker Arrants（回答）: ColabでRTX 6000を借りるためかなり費用を払いました。L2 modelは1 fold、1 seedに3時間かかり、厳しかったです。

もっとGPU efficientなtraining方法はあるはずです。15k pretrainingと25kの差は約0.2 CVで、pretraining timeがほぼ2倍になる割には小さな差でした。

Q4: Synthetic well generation code

萤火2018（質問）: well synthesis codeの詳細を共有できますか？ 私の方法はfull synthesis/pretrainingまで行かず、各training wellに`n_synth` versionsを作り、他featureは固定し、`h_gr = h_gr_sim + h_gr_noise`（`gr_noise`はrandom training wellから取得）だけ変更してtraining dataへ加えます。`gr_noise = gr_obs - gr_sim`です。

Tucker Arrants（回答）: もちろんです。local training codeからClaudeに切り出させたのでbuggyかもしれませんが、詳細はすべてあります。poolの「tuning」は非常に時間がかかり、targeted experimentを数回しかできなかったのでoptimalとは程遠いでしょう。notebookのsettingsがfinal modelで使ったものです。

https://www.kaggle.com/code/tuckerarrants/rogii-synth-generator?scriptVersionId=340488304